



Secure File Storage

S e g u r a n ç a d e S o f t w a r e



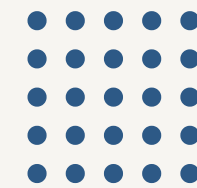
Rúben Silva pg57900

21 May 2026





Problem & Motivation



The Challenge:

- Os utilizadores armazenam ficheiros na cloud sem confiar no servidor
- O comprometimento do backend NÃO deve expor ficheiros
- Multi-Tenant deve impedir vazamentos entre utilizadores

Approach:

1. Full Privacy Mode
 - a. Client-side encryption
2. Secure Development lifecycle
3. Zero Knowledge:
 - a. O Backend só vê dados encriptados





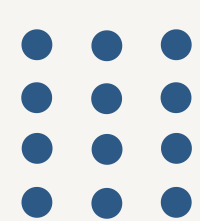
Req. do Enunciado

1. **Rest API Backend:** Fast API w/ SQLAlchemy
2. **CLI Client:** Python w/ AES-GCM encryption
3. **Auth (OIDC):** Google JWT validation via JWKS
4. **Multi-Tenancy & AuthZ:** DB-Level Tenant Isolation
5. **File Encryption (E2E):** AES-GCM with per-file DEKs
6. **Security Tests:** 13 Automated Security Tests
7. **Deployment:** Non-root container, env-based secrets – Docker
8. **Anonymous Link Sharing:** TTL-Protected file links
9. **File Versioning: Update** History per file
10. **Audit Logging:** Redacted Centralized logs

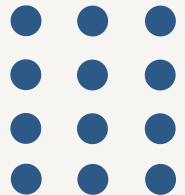




Defensive Security Engineering

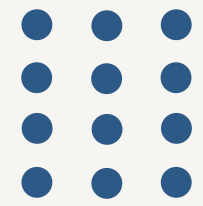


Stream-limited uploads	Unbounded RAM usage attacks	<code>_save_upload_stream_limited()</code> reads in 64KB chunks
Metadata rollback on failure	Dangling DB records (inconsistency)	If upload fails, delete metadata to stay consistent
Split auth/shared response	Authorization bypass via confusion	List returns <code>owned_files</code> and <code>shared_with_me</code> separately
Owner-first permission check	Cross-tenant fallback paths	Check owner first, then explicit share relation only
Bounded TTL validation	Invalid input (negative TTL, year 2099)	TTL must be 1-30 days, no exceptions
UUID filenames + encrypted content	Metadata leakage	Server never knows original filename
Per-request Rate Limiting	DoS exhaustion	60 reqs/min per IP, shared window
Cascade delete on file removal	Orphaned shares/links in DB	Foreign key <code>onDelete=CASCADE</code> cleans up
Strict nonce uniqueness	AES-GCM catastrophic reuse	Client generates new 96-bit random nonce per operation





Endpoints



- POST /file/ → **Upload Encrypted**
- GET /file/ → **List Owned** + *Shared Files*
- GET /files/{file_id}/download → **Download Specific File**
- PUT /files/{file_id} → **Update File** (*New Version*)
- DELETE /files/{file_id} → **Delete file** + *metadata cleanup*

- GET /files/{file_id}/permissions → **View Permissions**
- PUT /files/{file_id}/permissions → **View Permissions**

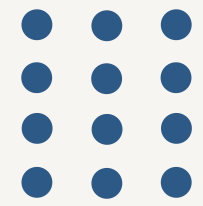
- POST /files/{file_id}/share → **Share** with other user
- POST /files/{file_id}/share/anonymous → **Create Anonymous Link** (TTL-Protected)
- GET files/anonymous/{link_id}/download → **Download via Anonymous Link**

- GET /health → **Saber se o server está vivo**
- POST auth/verify → **Verificar o OIDC JWT Token**





Endpoints



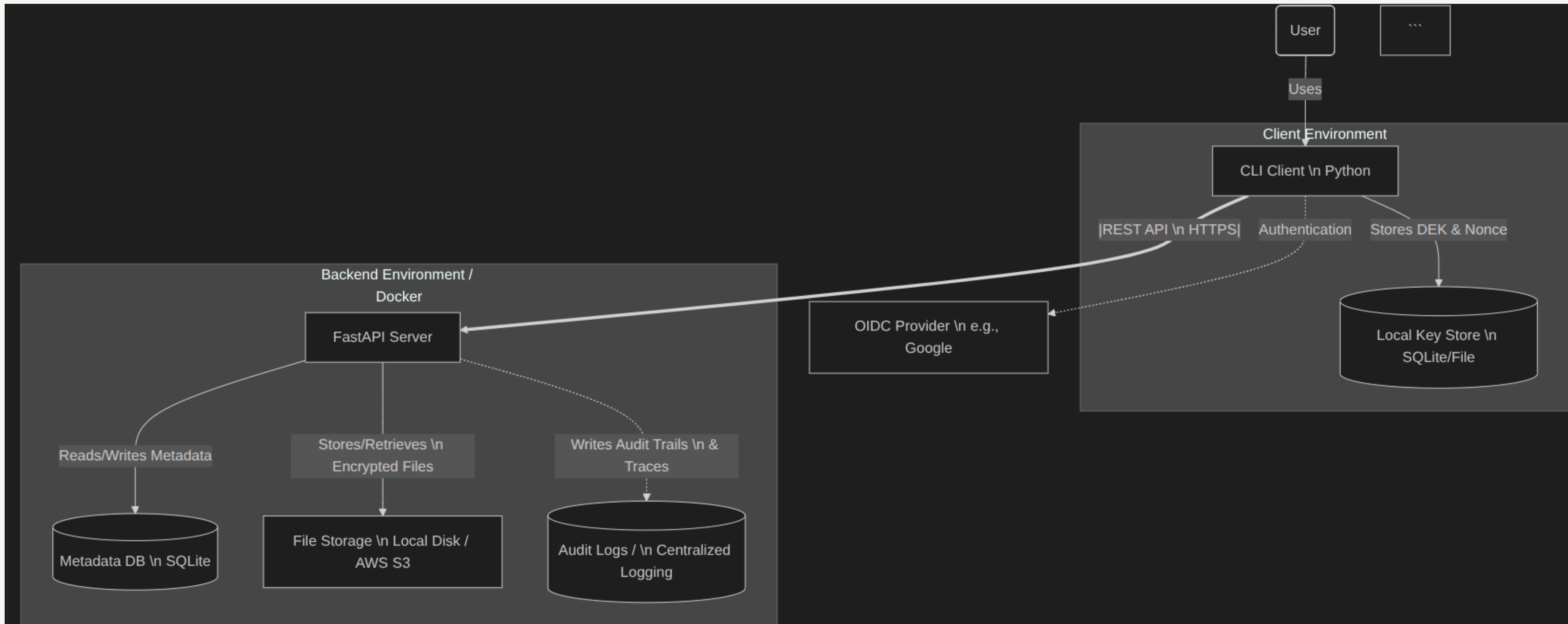
Defensive Features:

- **Apagar em cascata** (file → shares → links)
- **Metadata Rollback** (upload fail → metadata clean up)
- **Isolamento da Permissão** (Só ve o que tem direito! – share)
- **File Expiration Check** (404 se expirado, evita leaks)
- **Tenant ID Filtering** (owner == current_user)



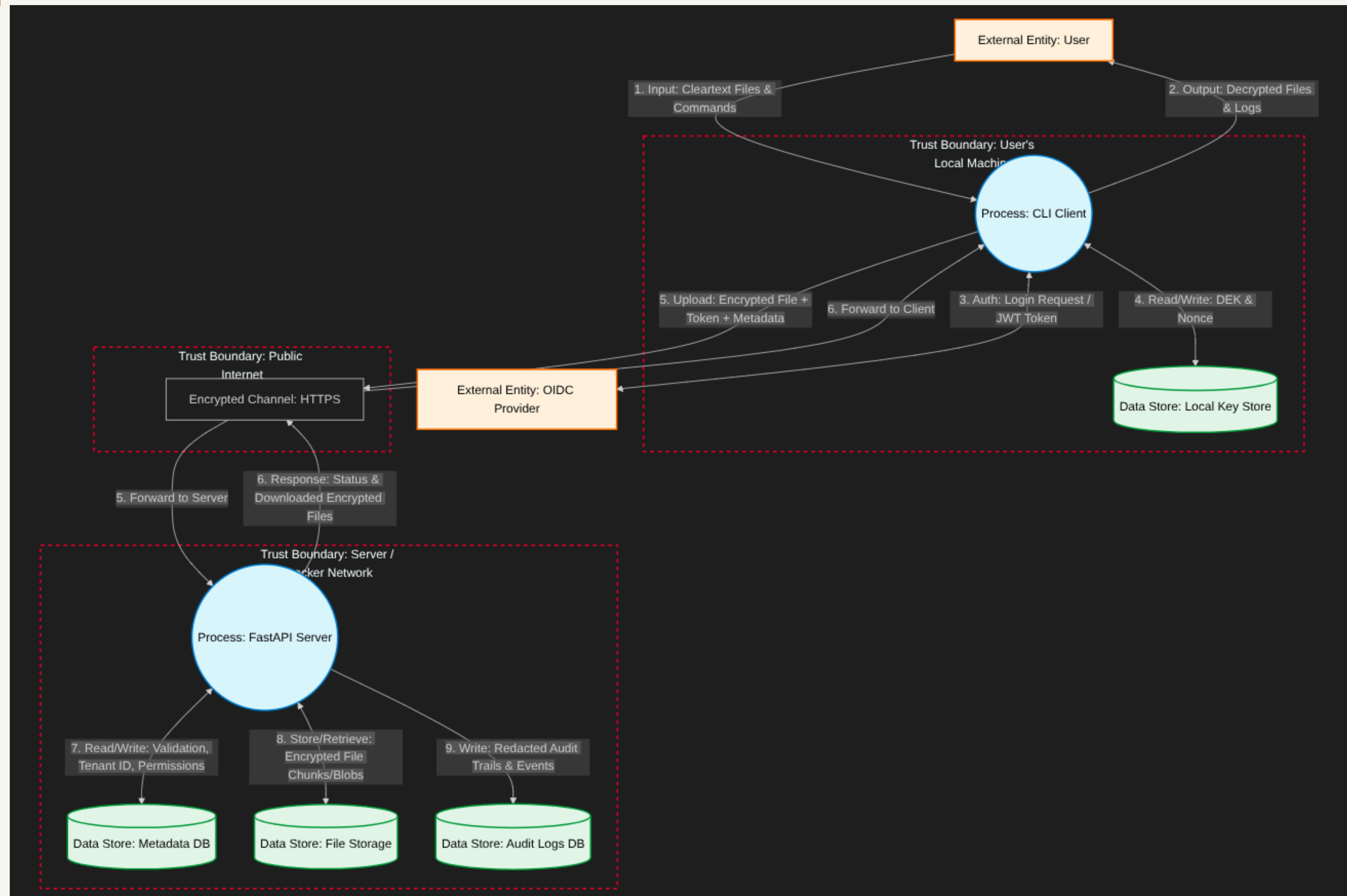


Arquitetura do Sistema





Data Flow Diagram - Threat Model





Threat Model - Stride



Threat Category	Potential Threat in Our System	Affected Component / Flow	Mitigation Strategy
Spoofing	An attacker intercepts or forges an OIDC token to impersonate a legitimate user and access their files.	Flow 3 & 5 (Public Internet)	Strict cryptographic validation of the OIDC JWT (signature, audience, and expiration) on the FastAPI server.
Tampering	A malicious actor or compromised network alters the encrypted file chunks in transit, or modifies the metadata directly in the SQLite database.	Flow 5 (Internet) & Flow 7 (Backend DB)	Use of TLS (HTTPS) for transit. Client-side encryption using AES-GCM ensures data integrity (tamper-evident). Use of SQLAlchemy ORM to prevent SQL Injection.
Repudiation	A user maliciously deletes a shared file and denies doing so, or support staff accesses files without traceability.	P2 (FastAPI Server)	Comprehensive centralized audit logging. All critical actions (upload, delete, permission changes) must be logged with a unique Request-ID , Timestamp, and the user's OIDC Subject ID.
Information Disclosure	The server or a cloud provider (AWS S3) inspects the file contents. Another tenant accesses metadata belonging to a different user.	Flow 8 (File Storage) & DS2 (Metadata DB)	Full Privacy Mode: Client-side encryption ensures the server only sees ciphertext. Strict Multi-Tenancy isolation logic ensures DB queries are always scoped to the authenticated user's Tenant ID. Filenames are anonymized (UUIDs).
Denial of Service (DoS)	An attacker spams the API with massive file uploads or thousands of requests, crashing the server or exhausting storage.	P2 (FastAPI Server) & DS3 (File Storage)	Implement API Rate Limiting and strict payload size limits for file uploads.
Elevation of Privilege	A standard user attempts to bypass permissions to access another tenant's files, or attempts to access the Admin Portal endpoints.	P2 (FastAPI Server)	Strict authorization checks on every endpoint. Role-Based Access Control (RBAC) implementation, ensuring admin features require specific OIDC roles.





Threat Model - Security Requirements

1. **Strict Tenant Isolation:** Every database query interacting with file metadata MUST include a filter for the authenticated user's Tenant ID. Cross-tenant data access is strictly forbidden unless explicitly granted via the sharing system.
2. **Zero-Knowledge Storage (Full Privacy Mode):** The server MUST NOT receive or store plaintext files. All files MUST be encrypted client-side using AES-GCM before transmission.
3. **Metadata Anonymization:** The server MUST NOT know the original filenames or extensions. Files must be stored on disk/S3 using randomly generated UUIDs.
4. **Strong Authentication & Authorization:** All API endpoints (except public share links) MUST require a valid OIDC JWT. Permissions (Read/Write) must be validated on every request.
5. **Secure Cryptographic Practices:** The client MUST ensure unique Nonce generation for every AES-GCM encryption operation to prevent catastrophic key reuse vulnerabilities.
6. **Input Validation & Safe File Handling:** All inputs MUST be sanitized. The server must be protected against Path Traversal and OS Command Injection by abstracting the file system layer and never using user-provided input for file paths.
7. **Secure Audit Logging:** The system MUST implement centralized logging. Logs MUST enforce redaction rules to ensure no secrets, PII, or tokens are ever leaked.
8. **Secure Deployment:** The Docker deployment MUST NOT contain hardcoded credentials. All secrets must be injected via environment variables (**ENV**), and containers MUST run as non-root users.





Escolha Criptográfica

AES-GCM:

- Confidentiality & Integrity
- key 256-bit
- Nonce 96-bit

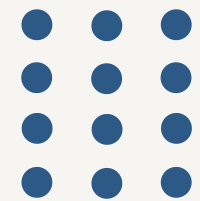
Per-File DEKs:

- Cada Ficheiro tem a sua chave pra evitar leaks que comprometam a BD toda de uma vez

Encriptação Envelopada:

- Usar um KEK, gerido por um KMS, para rodar as keys





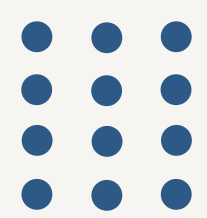
Automação de Testes de Sec.

Test	Validates
Test 1-2	Tenant isolation: User A cannot see User B's files
Test 3-4	Permission bypass: read-only users cannot update files
Test 5-6	Anonymous link expiration (TTL enforcement)
Test 7-8	File versioning: updates create new versions
Test 9	Oversized upload (>50 MB) rejected with 413
Test 10	Rate limiting blocks 61st request (60 req/min)
Test 11	Malformed JWT rejected (403 Forbidden)
Test 12	File deletion removes metadata + storage
Test 13	Audit log captures all critical actions



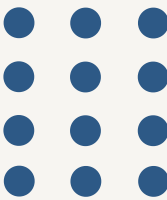


CI/CD



Pipeline:

1. **pytest** – 13 Security Tests
2. **bandit** – Static Analysis
3. **pip-audit** – Dependency Vulnerability Scan
4. **detect-secrets** – Credential Leaking





Deploy & Hardening

Deploy (secure by default)

```
FROM python:3.12-slim
RUN useradd -m -u 1000 secureuser # Non-root user
USER secureuser
ENV GOOGLE_CLIENT_ID=${GOOGLE_CLIENT_ID} # Secrets via ENV
```

Security Features:

- Non-root user
- env contém os secrets
- API corre na porta 8000 (non-privileged)





Secure File Storage

S e g u r a n ç a d e S o f t w a r e



Rúben Silva pg57900

21 May 2026

